TRAFFIC VIOLATION DETECTION AND EMERGENCY PRIORITY VECHICLE SYSTEM

A project report submitted to

Jawaharlal Nehru Technological University Kakinada, in the partial Fulfilment for the Award of Degree of

BACHELOR OF TECHNOLOGY IN

ARTIFICIAL INTELLIGENCE & DATASCIENCE

Submitted by

CH MAHENDRA VARMA	**22491A5408**
K.V. DILEEP VARMA	**23495A5401**
G. KOUSALYA	**22491A5412**
G. BRAMAIAH	**22491A5409**
U. SOWMYA	**22491A5455**
P. RAGHU VAMSI	**22491A5439**

Under the esteemed guidance of

Dr. M. Muthamizh Selvam

Assistant Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

QIS COLLEGE OF ENGINEERING AND *TECHNOLOGY*

(AUTONOMOUS)

An ISO 9001:2015 Certified institution, approved by AICTE & Reaccredited by NBA, NAAC 'A+' Grade Affiliated to Jawaharlal Nehru Technological University, Kakinada) VENGAMUKKAPALEM, ONGOLE – 523 272 , A.P

April -2026

QIS COLLEGE OF ENGINEERING AND *TECHNOLOGY*

(AUTONOMOUS)

An ISO 9001:2015 Certified institution, approved by AICTE & Reaccredited by NBA, NAAC 'A+'Grade

Affiliated to Jawaharlal Nehru Technological University, Kakinada) VENGAMUKKAPALEM,

ONGOLE – 523 272 , A.P

April 2026

DEPARTMENT OF ARTIFIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

This is to certify that the t*echnical report entitled* **TRAFFIC VIOLATION - DETECTION AND EMERGENCY PRIORITY VECHICLE SYSTEM is** *a bonafide work of the following final B Tech students in the partial fulfilment of the requirement for the award of the degree of bachelor of technology in ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for the academic year 2025-2026.*

CH. MAHENDRA VARMA	**22491A5408**
K.V. DILEEPVARMA	**23495A5401**
G. KOUSALYA	**22491A5412**
G. BRAMAIAH	**22491A5409**
U. SOWMYA	**22491A5455**
P. RAGHU VAMSI	**22491A5439**

Signature of the guide
Dr. M.Muthamizh Selvam, *M.Tech,(PhD)*
Assistant Professor

Signature of Head of Department
Dr. G. L. Vara Prasad, M. Tech, Ph.D
H*OD, Associate Professor in AIDS*

Signature of External Examiner

ACKNOWLEDGEMENT

We thank the almighty for giving us the courage and perseverance in completing the project. It is an acknowledgement for all those people for all those people who have given us their heartfelt cooperation in making in making the major project a grand success.

We would like to place on record our deep sense of gratitude to the Honorable Executive Chairman **Dr. N. S. Kalyan Chakravarthy**, Honorable Executive Vice Chairman **Dr. N. Sri Gayatri Devi** and Principal **Dr. Y.V.Hanumantha Rao** for providing the necessary facilities to carry out the project work.

We express our gratitude to the Head of the Department of **AIML&DS, Dr. G.** L.**Vara Prasad ,**Project Guid*e* **Dr. M. Muthamizh Selvam***,* and **Department faculty** for their valuable suggestions, guidance, and cooperation throughout the project.

We would like to express our thankfulness to **CSCDE & DPSR** for their constant motivation and valuable help throughout the project.

Finally, we would like to thank our Parents, Family and Friends for their cooperation in completing this project

Submitted by

CH MAHENDRA VARMA	22491A5408
K.V. DILEEP VARMA	23495A5401
G. KOUSALYA	22491A5412
G. BALA BRAMAIAH	22491A5409
U. SOWMYA LAKSHMI	22491A5455
P. RAGHU VAMSI	22491A5439

ABSTRACT

The Traffic Violation Detection and Emergency Priority System present an intelligent and automated traffic management framework that integrates embedded systems, Python-based computer vision, and IoT technologies to improve road safety and emergency response efficiency. The system employs an Arduino Mega 2560 to control multi-lane traffic signals, manage local indicators, and interface with alert devices, while an ESP32 microcontroller functions as an IoT gateway for real-time data transmission. A USB web camera captures live traffic video, which is processed using Python and a YOLOv8 deep learning model to detect traffic violations such as riding without a helmet and triple riding. Upon identifying a violation, the system triggers visual and audible alerts, displays violation information on an I2C LCD, and automatically transmits evidence images with timestamps to a predefined email account for monitoring and documentation. Simultaneously, vehicle count data from each lane is uploaded to the Thing Speak cloud platform to enable real-time traffic flow analysis. The system also incorporates an emergency priority mechanism through a dedicated pushbutton, allowing uninterrupted green signals for emergency vehicles. The proposed system demonstrates a scalable, cost-effective, and reliable solution for intelligent traffic control, automated violation detection, and emergency vehicle prioritization in urban traffic environments.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF TABLES	3
	LIST OF FIGURES	4
	LIST OF SYMBOLS AND ABBREVIATIONS	5
1	**INTRODUCTION**	
	1.1 Introduction	6
	1.2 Objective of the project	6
	1.3 Motivation of the Thesis	7
	1.4 Organisation of Thesis	8-9
2	**LITERATURE SURVEY**	
	2.1 Basic Concepts	10
	2.2 Related Work	10-17
	2.3 Research Gap	18

3	PROPOSED WORK AND ANALYSIS	
	3.1 Proposed System	19
	3.2 Feasibility Study	20
4	SYSTEM DESIGN (FOR CSE & ALLIED)/ /MATERIALS AND METHODS (FOR OTHERS)	
	4.1 System Design	21-22
	4.2 Block Diagram	23
	4.3 Modules	24-27
	4.4 UML Diagrams	27-32
5	IMPLEMENTATION/RESULTSAND DISCUSSION	
	5.1 Software Requirements	33
	5.2 Hardware Requirements	34
	5.3 Technologies used	35-36
	5.4 Coding	37-47
6	RESULTS/FUTURE SCOPE OF WORK	51-55
7	CONCLUSION	56
8	FUTURE ENHANCEMENTS	57-58
9	APPENDIX APPENDICES	60-65

10 REFERENCES 66-67

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1	Existing Methods	13-17

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1	Circuit connection	21
2	Block Diagram	23
3	Arduino MAGA 2560	24
4	LED Indicators	25
5	ESP 32 Controller	26
6	Buzzer	26
7	Sound Sensor	27
8	UML Diagram	28
9	UML Use Case Diagram	29
10	Prototype Set Up	52
11	Simulation of Traffic Lights	52
12	Ambulance Detection in Different ways	53
13	Identify Traffic Violation	53
14	Executed Python code	54

LIST OF SYMBOLS AND ABBREVIATIONS

INDEX	ABBREVIATION	DESCRIPTION
1	YOLOv8	You Only Look Once
2	IOT	Internet of Things

CHAPTER-1

INTRODUCTION

1.1 Introduction

Rapid urbanization and the exponential growth of vehicles have made effective traffic management and road safety enforcement critical challenges in modern cities. Conventional traffic control systems rely heavily on manual monitoring and fixed signal timings, which often fail to detect traffic violations in real time or provide timely priority to emergency vehicles. To address these limitations, the Traffic Violation Detection and Emergency Priority System introduce an intelligent, automated approach that combines embedded systems, computer vision, and IoT technologies. By leveraging deep learning–based video analysis for violation detection, real-time traffic data monitoring through cloud platforms, and priority control for emergency vehicles, the system aims to enhance road safety, reduce human intervention, and improve emergency response efficiency. This integrated solution provides a practical and scalable framework for smart urban traffic management.

1.2 Objective of the Project

The primary objective of the Traffic Violation Detection and Emergency Priority System is to design and develop an automated and intelligent traffic management framework that enhances road safety and operational efficiency. The system aims to implement real-time traffic violation detection using computer vision and deep learning techniques to identify offenses such as riding without a helmet and triple riding. It seeks to establish real-time traffic signal control through the Arduino Mega 2560 for efficient multi-lane management and alert operations. Another key objective is to integrate IoT- based communication using the ESP32 module to transmit realtime traffic data and violation information to cloud platforms for monitoring and analysis. The project also focuses on providing automated alert and documentation

mechanisms, including visual indicators, audible buzzers, LCD displays, and email notifications with captured violation evidence. Additionally, it aims to incorporate an emergency vehicle priority system that ensures uninterrupted green signals during emergency situations, thereby reducing response time. Furthermore, the system is designed to enable continuous traffic monitoring by uploading vehicle count data to cloud services such as ThingSpeak, while ensuring that the overall solution remains scalable, cost-effective, and reliable for deployment in smart city traffic management infrastructures.

1.3 Motivation of the Thesis

Road accidents and traffic violations remain one of the leading causes of fatalities and injuries in urban areas. A significant number of these incidents occur due to noncompliance with basic traffic rules, such as not wearing helmets or carrying multiple riders on two-wheelers. Existing enforcement methods rely heavily on human supervision, which is labor-intensive, error-prone, and ineffective for continuous realtime monitoring.

Another major concern is the lack of efficient traffic signal prioritization for emergency vehicles such as ambulances and fire engines. Delays caused by traffic congestion can result in loss of lives and property. Conventional systems often depend on sirens or manual signal overrides, which are unreliable in high-density traffic conditions. The motivation behind this project is to develop an **automated, intelligent, and real- time traffic management system** that reduces dependency on manual enforcement while improving road safety and emergency response efficiency. By leveraging artificial intelligence for violation detection and IoT for data communication, the system aims to provide faster decision-making, accurate monitoring, and effective traffic control. The project also supports the broader vision of smart cities by enabling data-driven traffic analysis and scalable deployment.

1.4 Organization of the Thesis Chapter

1.4.1 – Introduction:

This chapter presents a brief overview of the project titled **"**Traffic Violation Detection and Emergency Priority Vehicle System**."** It explains the background, objectives, motivation, scope, and significance of the proposed system. The chapter also highlights the limitations of traditional traffic monitoring methods and describes how the proposed system provides an automated solution for traffic management and emergency vehicle prioritization.

Chapter 1.4. 2 – Literature Review:

This chapter reviews prior work relevant to automatic traffic-violation detection and systems that provide priority to emergency vehicles at signalized intersections. The review is structured around (1) methods for detecting traffic violations, (2) technologies and strategies for emergency vehicle priority

Chapter 1.4. 3 – Proposed Work and Analysis:

The proposed system titled **"**Traffic Violation Detection and Emergency Priority Vehicle System**"** is designed to automate traffic monitoring and provide priority passage to emergency vehicles at signal intersections.The system integrates cameras, sensors, a microcontroller unit, and a traffic signal control mechanism to detect violations and manage traffic signals efficiently.

Chapter1.4. 4 – System Design:

This chapter describes the overall design of the Traffic Violation Detection and Emergency Priority Vehicle System. It explains the system architecture, module design, hardware components, software design, and data flow of the proposed system.

Chapter1.4. 5 – Implementation:

This chapter describes the practical implementation of the Traffic Violation Detection and Emergency Priority Vehicle System. It explains the hardware setup, software development, system integration, and testing procedures followed to build the working model.

Chapter 1.4.6 – Results and Discussion:

This chapter presents the results obtained after implementing and testing the Traffic Violation Detection and Emergency Priority Vehicle System. It also discusses the performance of the system, its effectiveness in real-time scenarios, and the observed limitations.

Chapter1.4. 7 – Conclusion:

This final chapter summarizes The Traffic Violation Detection and Emergency Priority Vehicle System was successfully designed and implemented to improve traffic management and road safety at signalized intersections.

Chapter1.4. 8 – Future Enhancements and Appendix:

This chapter discusses the possible future improvements to the proposed Traffic Violation Detection and Emergency Priority Vehicle System and provides supporting technical details in the appendix section.

Chapter1.4. 9- References:

This chapter contains references of our project research papers.

CHAPTER-2

LITERATURE REVIEW

2.1 Basic Concepts

A literature review is a critical and systematic analysis of previously published research works related to the selected project domain. It provides a comprehensive understanding of existing technologies, methodologies, and solutions proposed by various researchers, helping to identify current trends, research gaps, and limitations in the field. In the context of traffic management and violation detection systems, the literature review focuses on studies involving such as YOLO, IoT-based traffic control mechanisms, and emergency vehicle prioritization techniques. By reviewing earlier research, the strengths and weaknesses of conventional and modern traffic monitoring systems can be evaluated, including challenges related to manual enforcement, fixed signal timing, lack of real-time violation detection, and inefficient emergency response. The literature review also helps in justifying the need for the proposed system by highlighting how the integration of embedded systems, and real-time data analytics can overcome the shortcomings of existing approaches. Overall, the literature review forms a theoretical foundation for the project, guides system design decisions, and ensures that the proposed solution is innovative, relevant, and aligned with current research advancements.

2.2 Related Work

2.2.1Title: AI-Powered Helmet and Vehicle Number Plate Recognition System with Automatic Traffic Violation Detection and Chalan Generation

This work addresses road safety violations such as riding without helmets and triple riding using an automated computer vision system. It utilises the YOLOv8 object detection model to detect rider, helmet, and motorcycle objects in real time, and applies OCR for reading vehicle number plates. Once a violation is detected, the system automatically generates and sends an e-challan to the registered vehicle owner via email. The system achieves high detection accuracy under diverse lighting and weather

conditions, reduces manual monitoring effort, and enhances road safety enforcement. [1]

2.2.2Title: Traffic Eye: A Literature Survey on AI-Powered Traffic Rules Violation Detection System

This work highlights the growing use of AI, machine learning, and computer vision for automated detection of traffic rule violations. The review focuses on object detection models such as YOLO to identify violations like speeding, redlight jumping, triple riding, and helmet non-compliance, coupled with OCR for license plate recognition. The authors discuss advantages of automated systems over manual monitoring, including scalability, real-time performance, and reduced human error in urban environments. [2]

2.2.3Title: AI and IoT based Traffic Signal and Rule Enforcement System

This work proposes an intelligent traffic signal and rule enforcement framework that integrates IoT sensors and AI algorithms to monitor traffic densities and detect iolations in real time. The system uses sensor data and surveillanc cameras to detect instances of red-light running and over-speeding, and transmits real-time data to a central system for adaptive signal control and enforcement. It demonstrates improved traffic flow efficiency and violation monitoring over static signal timing systems. [3]

2.2.4Title: Real-Time Multi-class Helmet Violation Detection Using YOLOv8 with License Plate Recognition

This work proposes an end-to-end real-time system that detects motorcycles, helmet compliance status, and license plates from video streams. The approach uses the YOLO object detection framework and Easy OCR for license plate recognition, enabling multi-class detection including helmet compliance. The system is evaluated for its ability to capture varied traffic behaviors and demonstrates robust performance in real-time monitoring scenarios. [4]

2.2.5. Title: Vehicle Detection and Traffic Violation Parking Management System

This work designed to detect motorcycle-related traffic violations and manage parking spaces using deep learning and machine learning models. The authors have conducted an indepth analysis of major traffic issues such as helmet noncompliance, triple riding, vehicle detection, and vacant parking spot identification. They also examined limitations of manual enforcement and the need for intelligent automated monitoring systems. The methodology used involves YOLOv3 for realtime object detection, Haar Cascade Classifier for motorcycle/helmet-related recognition, Support Vector Classification (SVC) for license plate character recognition, and OCR for extracting license plate numbers. [5]

2.2.6. Title: Intelligent transportation and realtime traffic monitoring systems

The understanding of advanced vehicle detection techniques and their broader impact on intelligent transportation and realtime traffic monitoring systems. This paper explores modern deep learning–based detection methods, especially R- CNN and YOLO frameworks, used for detecting vehicles in traffic scenes. The authors have conducted an in-depth analysis of traditional methods, region-based approaches (R-CNN variants), and regression based methods (YOLO versions), comparing their strengths, weaknesses, and performance in complex environments. This provides valuable insights into how advanced deep learning models handle challenges such as multi-scale vehicles, occlusion, varying illumination, and diverse weather conditions. [6]

2.2.7. Title: multimodal sensing, including lane detection, driver posture and attention monitoring, pedestrian/vehicle detection

This work had conducted an in-depth analysis of dynamic context capture using multimodal sensing, including lane detection, driver posture and attention monitoring, pedestrian/vehicle detection, and driver intent prediction, providing valuable insights into how integrated vision systems improve situational awareness and reduce accident risk. The methodology used involves developing and evaluating algorithms for omnidirectional imaging, stereo vision, head-pose estimation, lane tracking, and Bayesian learning for intent prediction, and the results indicate that combining “looking-in” and “looking-out” sensing significantly improves accuracy in detecting hazards, predicting driver behavior, and supporting active safety systems. [7]

2.2.8. Title: computer vision–enabled intelligent surveillance systems designed to improve real-time urban traffic control

This work explores computer vision–enabled intelligent surveillance systems designed to improve real-time urban traffic control. The authors have conducted an in-depth analysis of AI-driven traffic monitoring, deep learning–based object detection, IoT-enabled sensing, adaptive signal control, violation detection, congestion prediction, and real-time analytics, providing valuable insights into how integrated AI-IoT systems can optimize traffic flow, reduce congestion, enhance road safety, and support smart city development. This study contributes to developing intelligent, scalable, and automated urban traffic management frameworks that support next-generation smart city infrastructure. [8]

2.2.9. Title: traffic violation alert generation system that uses image processing and deep learning to detect rule breaking vehicles

This work proposes a traffic violation alert generation system that uses image processing and deep learning to detect rule breaking vehicles. The authors focus on violations such as signal jumping, lane crossing, and unauthorized vehicle movement, aiming to automate the process of identifying offenders. The methodology involves using YOLO object detection, tracking vehicle movement, detecting violations, and extracting license plate information for alert generation. The results show that the system can successfully identify multiple types of violations and capture vehicle numbers accurately, improving the efficiency of traffic monitoring. This study contributes to developing smart surveillance systems for intelligent traffic management. [9]

2.2.10. Title: deep-learning-based approach for detecting traffic accidents using video data

This work explores a deep-learning-based approach for detecting traffic accidents using video data. The authors have conducted an in-depth analysis of CNN-based feature extraction, accident recognition, and real-time video processing, providing valuable insights into how temporal and spatial features can be effectively used to identify abnormal traffic [10]

Tab1e 1. Existing Method

REF NO.	TITLE	AUTHOR	TECHNIQUE USED	PROS	CONS
1	Vehicle Detection and Traffic Violation Parking Management System.	Ashwini Phalke, Vaishnavi Hambir, Namrata Utekar, Sudharshni Nadar.	CCTV + AI-based Traffic Monitoring, YOLOv3, SVC, OCR , Machine Learning–based Parking Space Detection	Automates traffic violation detection and parking management without manual monitoring.	Accuracy can drop in poor lighting, occlusion, or low-resolution CCTV footage.
2	A Review on Advanced Detection Methods in Vehicle Traffic Scenes.	NyThuzMyatein Chan, ar Tint.	CCTV + Deep Learning–based Traffic Monitoring, RCNN Family, YOLO Framework, **CNN-based Feature Extraction,ITS**	Deep learning models provide high accuracy and robustness in complex traffic scenes.	High computational and GPU requirements limit deployment on low-cost systems.

3	Looking-In and LookingOut of a Vehicle: ComputerVisionBased Enhanced Vehicle Safety.	Mohan Manubhai Trivedi, Tarak Gandhi, Joel McCall	CCTV + Computer Vision, Deep Learning, AI Traffic Monitoring, IoT Sensors, Adaptive Signals	Combining looking-in and looking-out vision significantly improves situational awareness and safety.	System complexity and computational cost increase due to multiple sensors and algorithms.
4	Computer Vision Enabled Smart Surveillance for Urban Traffic Control.	P. Siva, Garige Bhavani Pujitha, Gogula Siva Krishna, Gadde Hemanth	CCTV + Computer Vision, Deep Learning Detection, AI Traffic Monitoring, IoT Sensors, Adaptive Signals, Violation Detection, Congestion Prediction, Smart City Traffic System	Integrated AI– IoT systems enable efficient, automated, and scalable urban traffic control.	High deployment cost and infrastructure complexity limit implementation in smaller cities

5	Adaptive Traffic Light Controller Based on Congestion Detection Using Computer Vision	R.G.Putra, W.Pribadi, I.Yuwono, D. E. J. Sudirman	CCTV + Computer Vision, YOLOv3 Object Detection, Vehicle Density Estimation, Adaptive Traffic Signal Control, AI based Congestion Detection	Reduces congestion by dynamically adjusting green-light timing.	Performance depends on camera quality and lighting conditions.
6	Traffic Violations Alert Generation Using Image Processing	S. Narmatha, R.Rajalakshmi.	CCTV + Image Processing, YOLO Object Detection, Vehicle Tracking, Traffic Violation Detection, License Plate Recognition	Automates detection of multiple traffic violations with higher efficiency.	Accuracy may decrease in crowded scenes or occluded vehicles.

7	Traffic Accident Detection Based on Deep Learning	Xin Shen, Junyi Li, Fan He, Pengcheng Guo	CCTV + Deep Learning, CNN-based Feature Extraction, Optical Flow Analysis, Realtime Video Processing, AIbased Accident Detection	Detects affic accidents ly automatical in with high accuracy real time	Requires high computation power for continuous video analysis
8	Intelligent Vehicle Detection and Tracking for Highway Driving	Wanxin Xu, Meikang Qiu, Zhi Chen, Hai Su	CCTV + Computer Vision, HOG + SVM, Partbased Model + Latent SVM, Mean Shift Tracking	Provides accurate and reliable vehicle detection and tracking on highways.	High computation cost due to partbased and latent SVM models. .

9	Acoustic-Based Emergency Vehicle Detection Using Convolutional Neural Networks	Van-Thuan Tran, Wei-Ho Tsai	Audio Sensors + Deep Learning, CNN (1D & 2D), MFCC & Log-Mel Features, Siren Detection AI	Detects emergency vehicles with very high accuracy using siren sounds.	Performance may degrade in noisy urban environments.
10	Smart Navigation System for Emergency Vehicles (SNSEV): Utilizing Fog and Cloud Computing Technology for Real-Time Traffic Management	Fatma M. Talaat, Samah A. Gamel	IoT Sensors, Fog Computing, Cloud Computing, Real-time Traffic Analytics, Smart Routing	Reduces emergency vehicle delay through realtime optimized routing.	Depends on reliable network connectivity and infrastructure.

2.3 RESEARCH GAP:

Most of these systems focus primarily on violation detection and automated Echallan generation without integrating intelligent traffic signal control mechanisms. Existing solutions generally operate as standalone monitoring systems and do not combine real-time violation detection with adaptive multi-lane traffic management. Furthermore, very few studies incorporate an emergency vehicle priority mechanism to dynamically override traffic signals and provide uninterrupted green signals, which is critical for reducing emergency response time. Many of the proposed systems also depend on high computational resources such as GPUs, limiting their scalability and cost-effectiveness for widespread smart city deployment. In addition, several models lack robust cloud-based monitoring and real-time traffic data analytics, restricting centralized supervision and long-term traffic pattern analysis. Environmental factors such as lighting conditions, weather variations, and dataset dependency further affect detection accuracy in practical urban environments. Therefore, there exists a significant need for a unified, scalable, and cost-effective system that integrates real-time violation detection, intelligent traffic signal control, IoT-based cloud monitoring, and emergency vehicle prioritization within a single embedded framework.

CHAPTER-3

PROPOSED WORK AND ANALYSIS

3.1 Proposed System

proposed Traffic Violation Detection and Emergency Priority System introduce an intelligent and automated framework by integrating embedded controllers, deep learning-based computer vision, and IoT technologies. The system employs a USB camera and a Python-based YOLOv8 model to perform real-time detection of traffic violations such as riding without a helmet and triple riding, enabling immediate and accurate enforcement. An Arduino Mega 2560 controls traffic signal operations, alert mechanisms, and display units, while an ESP32 module facilitates wireless data transmission to the Thing Speak cloud for real-time traffic monitoring and analysis. Upon violation detection, the system generates instant visual and audible alerts and automatically sends violation evidence to a designated email account for documentation. Additionally, an emergency priority mechanism allows dynamic signal control to provide uninterrupted green signals for emergency vehicles, significantly improving response times. This proposed model offers a scalable, cost-effective, and intelligent alternative to traditional traffic systems by combining automation, real- time analytics, and cloud-based monitoring.

3.2 Feasibility Study

Technical Feasibility: The proposed Traffic Violation Detection and Emergency Priority System is technically feasible as it utilizes well-established and widely supported technologies. The system integrates an Arduino Mega 2560 for traffic signal control, an ESP32 microcontroller for IoT-based communication, and a USB camera for real-time video capture. Python- based computer vision and deep learning techniques using the YOLOv8 model enable accurate detection of traffic violations such as helmet noncompliance and triple riding. All hardware components are easily available in the market and compatible with standard development tools like Arduino IDE and Python libraries. Cloud platforms such as ThingSpeak support real-time data storage and analysis, making the system reliable and scalable. Therefore, the project can be implemented successfully using existing technical resources and skills.

Operational Feasibility: The system is operationally feasible as it is designed for ease of use and minimal human intervention. Once deployed, the system operates automatically by monitoring traffic, detecting violations, controlling signals, and generating alerts without requiring continuous manual supervision. Traffic authorities can access violation data and traffic statistics remotely through cloud services, reducing workload and improving efficiency. The emergency priority mechanism is simple to operate using a pushbutton, ensuring quick activation during emergency situations. Since the system follows existing traffic management workflows, it can be smoothly integrated into real-world traffic environments without major operational changes.

Economic Feasibility: The proposed system is economically feasible as it uses low-cost and readily available hardware components such as Arduino, ESP32, LEDs, buzzers, and USB cameras. Open-source software platforms like Arduino IDE, Python, and deep learning frameworks significantly reduce development and licensing costs. Automation of traffic violation detection minimizes the need for manual monitoring and enforcement, leading to long-term cost savings for traffic authorities. Additionally, the system's scalability allows gradual deployment across multiple locations, making it a cost- effective solution suitable for smart city applications.

Schedule Feasibility:

The project is schedule feasible as it can be completed within the allocated time frame using a structured development approach. The project tasks can be divided into phases such as requirement analysis, hardware setup, software development, system integration, testing, and documentation. Each phase can be executed sequentially or in parallel, ensuring timely completion. Since the technologies and components used are familiar and supported by extensive documentation, development delays are minimal. Therefore, the project can be successfully completed within the planned academic schedule.

CHAPTER-4

SYSTEM DESIGN

4.1 System Design

The system design of the Traffic Violation Detection and Emergency Priority System is based on an integrated embedded and IoT architecture that combines sensing, processing, control, and alert mechanisms. At the core of the system is the Arduino Mega 2560, which functions as the main control unit responsible for traffic signal sequencing, input handling, and output control. Multiple LEDs connected to the Arduino represent traffic signals for different lanes, while a buzzer provides audible alerts during traffic violations or emergency conditions. A 16×2 I2C LCD module is interfaced with the controller to display real-time system messages such as traffic status, lane information, and violation alerts. The pushbutton module is used to activate the emergency priority mode, allowing uninterrupted green signals for emergency vehicles. An ESP32 module serves as the IoT communication gateway, enabling wireless transmission of traffic data and system status to cloud platforms for monitoring and analysis. The system is powered through a regulated power supply to ensure stable operation of all components. The overall design ensures coordinated interaction between hardware modules, enabling real-time traffic control, automated alerts, emergency prioritization, and scalable smart traffic management functionality.

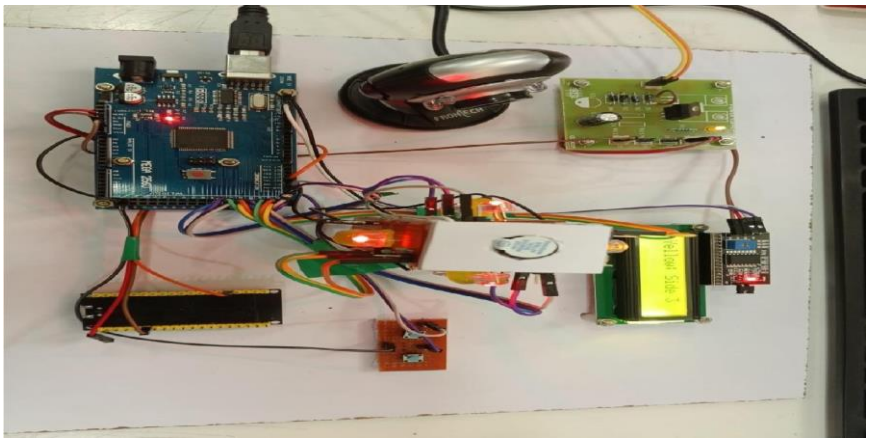

Fig 4.1.1: Circuit connection

In this figure 4.1.1 The circuit is designed around the Arduino Mega 2560, which acts as the central control unit and interfaces with all peripheral modules. The Arduino Mega is powered through a USB cable, which supplies regulated 5V required for logic-level operation. The GND pins of all modules are connected to a common ground to ensure proper signal reference and stable operation.

The traffic signal LEDs (Red, Yellow, and Green) are connected to the Arduino's digital output pins through current-limiting resistors. Each LED represents a traffic signal state for a specific lane. The Arduino controls these LEDs by setting the corresponding digital pins HIGH or LOW based on the programmed traffic sequence and emergency conditions.

A buzzer module is connected to one of the Arduino's digital output pins. When a traffic violation is detected or when an emergency mode is activated, the Arduino drives this pin HIGH, causing the buzzer to generate an audible alert. The buzzer's negative terminal is connected to ground.

The 16×2 LCD display with I2C interface is connected to the Arduino Mega using the I2C communication protocol. The LCD's SDA (data) and SCL (clock) pins are connected to the Arduino Mega's SDA (pin 20) and SCL (pin 21) respectively. The LCD is powered using the 5V and GND pins from the Arduino. This LCD displays system status messages such as lane information, traffic signal states, and violation alerts.

An emergency pushbutton switch is connected to a digital input pin of the Arduino Mega. One terminal of the pushbutton is connected to the input pin, while the other terminal is connected to ground. The internal pull-up resistor of the Arduino is enabled in software, so when the button is pressed, the input pin reads LOW, triggering the emergency priority mode in which the traffic signal turns green for uninterrupted vehicle passage.

The ESP32 module is interfaced with the Arduino Mega using serial communication.

The ESP32's TX and RX pins are connected to one of the Arduino Mega's hardware serial ports (such as Serial1 or Serial2), ensuring reliable data transfer. The ESP32 is powered using a regulated 5V or 3.3V supply (as required), with common ground shared with the Arduino. This module transmits traffic data and system updates to cloud platforms for real-time monitoring.

All components receive power from a regulated power supply, ensuring stable voltage levels and preventing damage to sensitive modules. The modular wiring approach allows easy

troubleshooting and future expansion. Overall, the circuit connections enable seamless coordination between sensing, control, alert generation, display, and IoT communication, ensuring reliable operation of the Traffic Violation Detection and Emergency Priority System.

4.2 BLOCK DIAGRAM

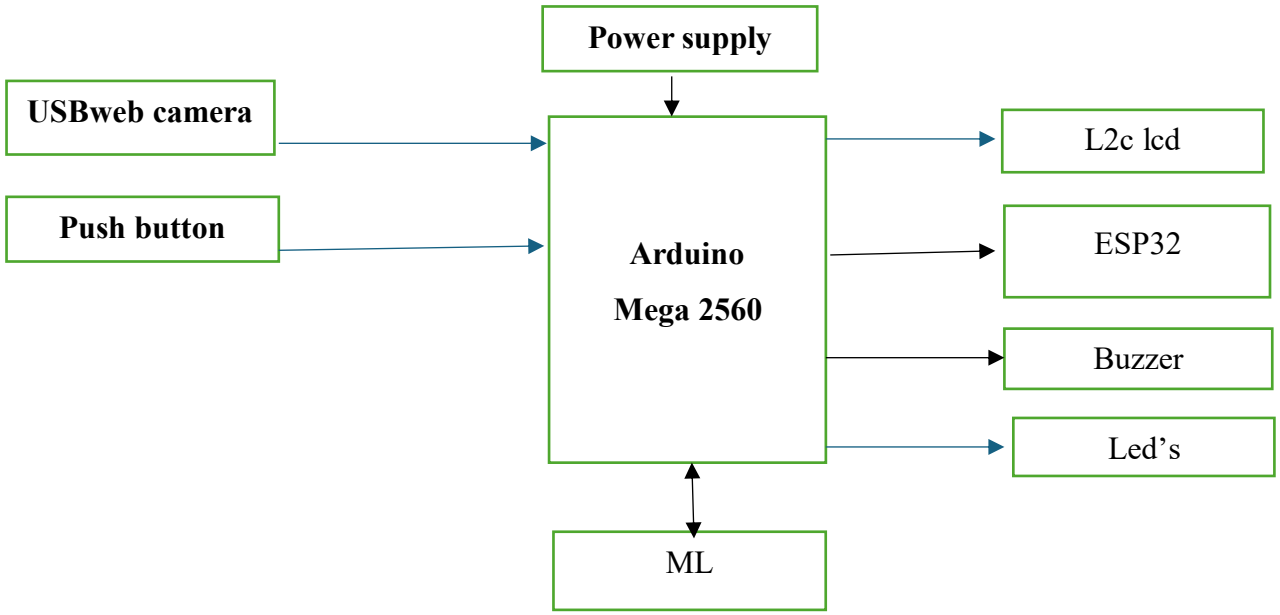

Fig 4.1.2: Block Diagram for traffic violation detection and emergency vechicle priority system.

Explanation for the Block Diagram:

The given block diagram in Fig 4.1.2 represents an Arduino Mega2560–based embedded system designed to collect input data, process it, and generate appropriate outputs. The power supply provides regulated electrical power to all components, ensuring stable system operation. The input section consists of a USB web camera and a pushbutton. The USB web camera captures real-time

visual data, which can be used for applications such as monitoring or detection, while the pushbutton allows manual user control for functions like start, stop, or reset. These input signals are sent to the Arduino Mega2560, which acts as the main controller of the system. The Arduino processes the received data according to the programmed logic and coordinates system operations. An ESP32 microcontroller is integrated with the Arduino to enable wireless communication and IoT functionality, allowing data transmission or remote monitoring. Based on the processing results, the Arduino controls the output devices, which include an I2C LCD for displaying system status and messages, a buzzer for audio alerts, LEDs for visual indication of different operating states, and a motor for performing physical or mechanical actions. Overall, the system follows a structured flow in which input data is acquired, processed by the microcontroller, and converted into meaningful visual, audio, and mechanical outputs, making it suitable for automation and real-time monitoring applications.

4.3 Modules

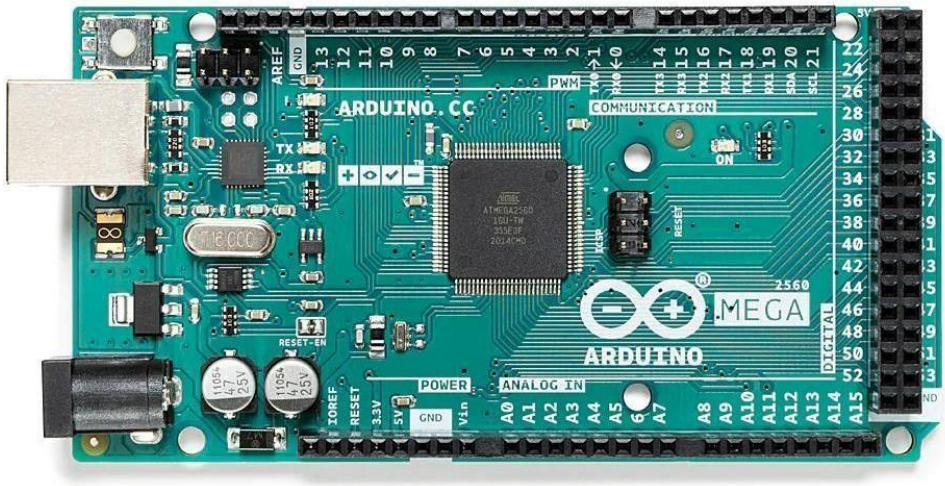

Fig 4.3.1: ARDUINO MEGA 2560

Fig 4.3.2LED Indicators

In our project, the Arduino Mega 2560 functions as the central control and processing unit, responsible for coordinating all system operations. It receives input signals from devices such as the USB web camera and pushbutton, processes the data according to the programmed logic, and generates control signals for various output devices. Among these outputs, LED lights play an important role as visual indicators of the system's operating status. The Arduino Mega 2560 controls the LEDs through its digital output pins, switching them ON or OFF to represent different conditions such as power status, detection results, alerts, or fault indications. The large number of I/O pins and higher memory capacity of the Arduino Mega 2560 allow it to manage multiple LEDs along with other peripherals like the LCD, buzzer, motor, and ESP32 module simultaneously. By combining the processing capability of the Arduino Mega 2560 with the quick response and low power consumption of LED lights, the system provides clear, real-time visual feedback, improving user awareness, system monitoring, and overall reliability of the project.

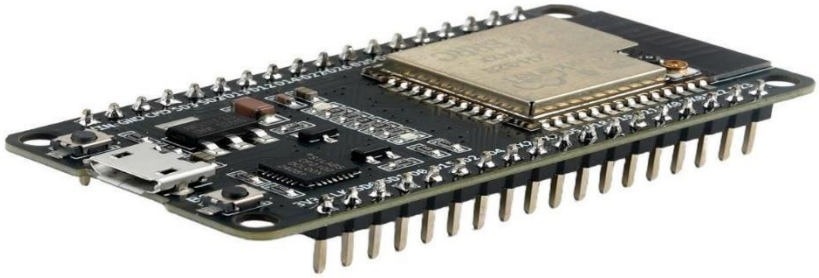

Fig .4.3.3 ESP 32 controller

In the Fig 4.3.3 ESP32 controller is used to provide wireless communication and IoT functionality. While the Arduino Mega 2560 performs the main control and processing tasks, the ESP32 enables the system to connect to Wi-Fi and Bluetooth networks, allowing data transmission to cloud platforms, mobile applications, or remote monitoring systems. It acts as a communication bridge between the hardware system and the internet. The ESP32 is chosen because it has a powerful dual-core processor, built-in Wi-Fi and Bluetooth modules, low power consumption, and multiple GPIO pins for interfacing. By integrating the ESP32, our system can send real-time alerts, upload detection data, and support remote control operations, making the project smarter, scalable, and suitable for IoT-based automation applications.

Fig .4.3.4 BUZZER

In the Fig 4.3.4 buzzer is an electronic audio signaling device that produces sound (beep or alarm) when electricity is applied. It is commonly used in electronic circuits and embedded systems like Arduino and IoT projects.

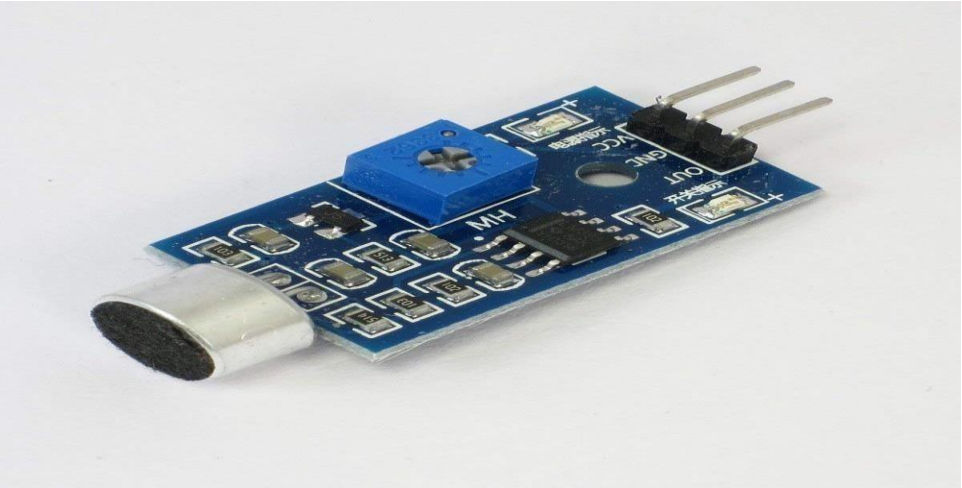

Fig. 4.3.5 Sound Sensor

4.4 UML Diagrams

UML (Unified Modeling Language) diagrams play a crucial role in project documentation by providing a clear, standardized, and visual representation of the system's design and functionality. They help bridge the gap between technical and non-technical stakeholders by illustrating how different components of the system interact, how data flows, and how processes are structured. For example, class diagrams show the static structure of the system, including objects, attributes, and relationships, while sequence diagrams depict the dynamic behavior by mapping out the order of interactions between components over time. This visual clarity makes it easier to identify potential design flaws, optimize system architecture, and ensure that everyone involved in the project—from developers to clients—has a shared understanding of how the system works. Including UML diagrams in a report not only enhances the professionalism of the documentation but also supports better planning, development, and maintenance of the system. They serve as a blueprint that guides implementation and future upgrades, making them an indispensable part of any well- documented software or hardware project.

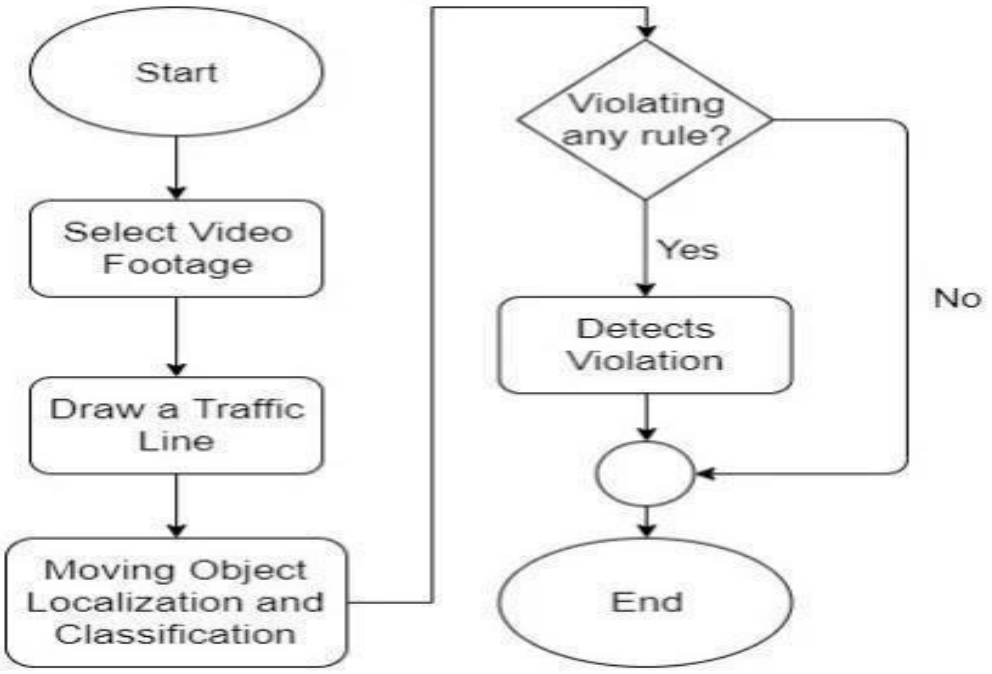

Fig 4.4.1: UMLActivity Diagram

In the Fig 4.4.1 shows the working of a traffic violation detection system in a simple flow. First, the process begins by selecting video footage from a traffic camera. Then, a traffic line (stop line) is drawn to define the boundary on the road. After that, the system detects and classifies moving objects such as vehicles. Once the vehicles are identified, the system checks whether any rule is being violated, like crossing the stop line. If a violation is detected, it records the violation; if not, the process continues without any action. Finally, the process ends after checking all conditions.

UMLActivity Diagram: Traffic Violation Detection

1. Automatic Traffic Monitoring

The system continuously monitors traffic at the signal junction without human intervention, reducing the need for manual traffic police supervision.

2. Accurate Violation Detection

By using video processing and object detection, the system accurately identifies traffic violations such as signal jumping, stop-line crossing, and rule breaking.

3. Real-Time Alerts

Visual (LED) and audio (buzzer) alerts are generated immediately when a violation is detected, enabling quick response.

4. Improved Road Safety

Automated enforcement discourages traffic rule violations, leading to safer roads and reduced accident rates.

5. Emergency Vehicle Priority

The system automatically detects emergency vehicles and gives them priority at traffic signals, ensuring faster and smoother movement during emergencies.

6. Reduced Human Error

Since detection and decision-making are automated, errors caused by human judgment are minimized.

7. Time and Cost Efficient

Continuous automated monitoring reduces manpower requirements and operational costs in the long run.

8. Scalable and Upgradable

The system can be easily upgraded to detect additional violations or integrated with cloudbased monitoring systems using ESP32.

9. 24/7 Operation

Unlike manual monitoring, the system can operate continuously throughout the day and night.

10. Smart City Compatibility

The system supports IoT integration and is suitable for smart traffic management and smart city applications.

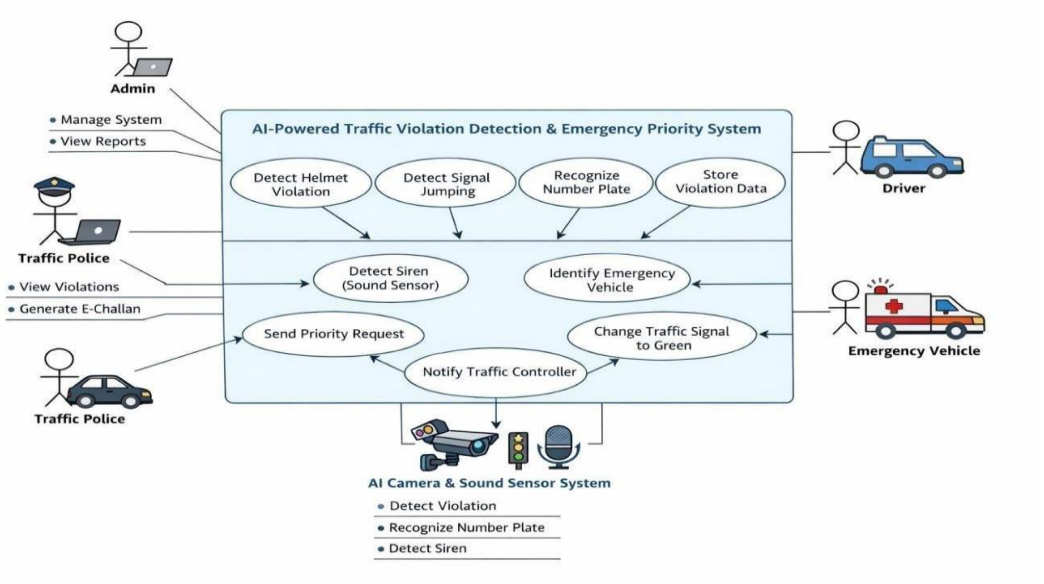

Fig 4.4.2: UML Use Case Diagram

Use Case Diagram for Traffic Violation Detection and Emergency priority :

The Use Case Diagram represents the functional behavior of the AI-based traffic management system. It shows how different actors interact with the system to perform traffic monitoring and emergency handling operations.

The system mainly consists of two major modules:

1. Traffic Violation Detection Module The AI Camera automatically

The AI camera system automatically monitors traffic and detects violations in real time. It identifies riders who are not wearing helmets and detects vehicles that jump traffic signals. When a violation occurs, the system captures and stores the relevant data for further review and enforcement. This automated monitoring helps improve road safety, supports effective traffic management, and assists authorities in maintaining traffic discipline.

The Traffic Police can:

Traffic police can access the system to view recorded traffic violations captured by the AI camera. Based on the detected violations, they can verify the details and generate an **E-Challan**

for the offender. This process helps authorities take quick action against traffic rule violations and ensures better enforcement of traffic regulation

The Traffic Police can:

Traffic police can access the system to view recorded traffic violations captured by the AI camera. Based on the detected violations, they can verify the details and generate an **E-Challan** for the offender. This process helps authorities take quick action against traffic rule violations and ensures better enforcement of traffic regulations.

The Driver:

The driver receives a notification when a traffic violation is detected by the system. The notification includes details about the violation and the generated E-Challan. After receiving the alert, the driver is required to pay the fine through the appropriate payment method within the given time. This process ensures accountability and encourages drivers to follow traffic rules.

The admin:

The admin is responsible for managing the overall system and ensuring its proper functioning. They monitor system operations, maintain records, and oversee the performance of the AI traffic monitoring system. The admin can also view reports related to traffic violations and system activities, which helps in analysing data and improving traffic management.

2. Emergency Priority Vehicle Module

The Sound Sensor:

The Emergency Priority Vehicle Module is designed to assist emergency vehicles such as ambulances, fire trucks, and police vehicles in reaching their destination quickly. The system uses a sound sensor to detect the siren sound of an approaching emergency vehicle. Once the siren is detected, the system analyses and confirms the presence of the emergency vehicle. After identification, it automatically sends a priority request to the traffic signal control system. The traffic signal then changes to green to provide a clear path for the emergency vehicle. At the same time, the traffic controller is notified to monitor and manage the situation effectively, ensuring smooth and safe traffic flow.

The Emergency Vehicle:

The emergency vehicle receives signal priority during emergency situations to ensure a faster and uninterrupted journey. When the system detects the siren of an approaching emergency vehicle, the traffic signal automatically changes to green. This allows the vehicle to pass through the intersection without delay, helping emergency services such as ambulances, fire trucks, and police vehicles reach their destination quickly and safely.

CHAPTER-5

IMPLEMENTATION

5.1 Software Requirements

The Traffic Violation Detection and Emergency Priority System uses multiple software tools and technologies, each responsible for a specific function within the system. The individual components are explained below:

1 Arduino IDE

Arduino IDE is an open-source development environment used to write, compile, and upload programs to the Arduino Mega 2560. It provides an easy interface for embedded system programming and serial communication. In this project, it is used to develop and upload control logic for traffic signals, LED indicators, buzzer alerts, I2C LCD display, and push-button-based emergency override.

2 Embedded C

Embedded C is the programming language used inside the Arduino IDE to control hardware components. It manages digital input/output pins, timing operations, traffic light sequencing, buzzer activation, LCD data display, and emergency signal switching. Embedded C ensures real-time control and reliable execution of signal operations.

3 Python Programming Language

Python is used for implementing the computer vision and deep learning components of the system. It processes live video input from the USB webcam and executes object detection algorithms. Python is selected because of its strong support for AI libraries and ease of integration with hardware modules.

4 YOLOv8 (Deep Learning Model)

YOLOv8 (You Only Look Once – Version 8) is a real-time object detection model used to identify motorcycles, riders, and helmet usage. It detects violations such as riding without a helmet and triple riding from live video frames. The model provides high detection accuracy and fast processing suitable for real-time applications.

5 OpenCV Library

OpenCV (Open-Source Computer Vision Library) is used in Python for video capture and image processing. It handles webcam input, frame extraction, image resizing, and display of detection results. OpenCV acts as the bridge between the camera hardware and the YOLO model.

6 ESP32 IoT Programming

The ESP32 microcontroller is programmed to transmit traffic data to the cloud platform. It uses Wi-Fi communication protocols to send vehicle count and violation statistics to Thing Speak for real-time monitoring and analytics.

7 Thing Speak Cloud Platform

Thing Speak is an IoT cloud platform used for storing and visualizing traffic data. It allows real-time monitoring of lane vehicle counts and system status. The platform enables data analytics and remote supervision of traffic flow.

8 Email Integration (SMTP Protocol)

The system uses Python's SMTP libraries to automatically send violation evidence images with timestamps to a predefined email account. This ensures documentation and reduces manual intervention in enforcement.

5.2 Hardware Requirements

The Traffic Violation Detection and Emergency Priority System integrates multiple hardware components to perform traffic monitoring, signal control, violation detection, and emergency prioritization. The individual components are explained below:

1 Arduino Mega 2560

The Arduino Mega 2560 is the main microcontroller of the system. It is based on the ATmega2560 and contains 54 digital input/output pins and 16 analog input pins. In this project, it controls the traffic signals (Red, Yellow, Green LEDs), buzzer alerts, I2C LCD display, and emergency push-button mechanism. It manages signal timing and coordinates communication with other modules.

2 ESP32 Microcontroller

The ESP32 is a low-cost Wi-Fi-enabled microcontroller used for IoT communication. It transmits real-time traffic data such as vehicle count and system status to the Thing Speak cloud platform. It enables wireless connectivity and remote monitoring of traffic statistics.

3 USB Web Camera

The USB webcam captures live traffic video. The video feed is processed using Python and the YOLOv8 deep learning model to detect traffic violations such as riding without a helmet and triple riding. It acts as the visual input device for the system.

4 I2C LCD Display (16x2)

The 16x2 LCD display is used to show system status and violation messages. It displays information such as detected violations, vehicle count, and emergency mode activation.

The I2C interface reduces the number of connecting wires required. **5 LEDs (Light Emitting Diodes)**

- Red LEDs – Indicate Stop signal
- Yellow LEDs – Indicate Ready/Wait signal
- Green LEDs – Indicate Go signal

Four sets of each LED are used to represent multi-lane traffic signals.

6 Buzzer

The buzzer provides an audible alert when a traffic violation is detected. It activates immediately upon detection to alert nearby authorities or users.

7 Push Buttons (2 Units)

Push buttons are used for manual control, especially for activating the emergency vehicle priority system. When pressed, the system overrides normal traffic signals and provides an uninterrupted green signal.

8 Power Supply Unit

The power supply provides regulated DC voltage to the system components. It includes a transformer, rectifier, filter capacitors, and voltage regulators (such as 7805 and 7812) to ensure stable 5V and 12V outputs for microcontrollers and peripheral devices.

9 Voltage Regulators (7805 / 7812)

These regulators ensure constant output voltage.

- 7805 → Provides regulated 5V
- 7812 → Provides regulated 12V

They protect the circuit from voltage fluctuations.

1 Supporting Components

- Arduino USB Cable – For programming and power supply

- Connecting Wires – For circuit connections
- Resistors – For LED current limiting
- Breadboard / PCB – For circuit assembly

5.3 Technologies used

1. Deep Learning (YOLOv8)

The system uses the YOLOv8 (You Only Look Once – Version 8) deep learning model for real-time object detection. It detects helmet usage and identifies violations such as riding without a helmet and triple riding. YOLOv8 provides high-speed and highaccuracy detection, making it suitable for real-time traffic monitoring applications.

2. Image Processing (OpenCV)

OpenCV (Open Source Computer Vision Library) is used for video frame capture, image processing, object tracking, and displaying detection results. It acts as an interface between the webcam hardware and the deep learning model implemented in Python.

3. Embedded Systems

Embedded systems technology is implemented using the Arduino Mega 2560 microcontroller. Embedded C programming is used to control traffic signal operations, LED indicators, buzzer alerts, LCD display messages, and emergency push-button mechanisms. This ensures reliable and real-time hardware control.

4. Internet of Things (IoT)

The system uses IoT technology through the ESP32 microcontroller for wireless communication. Traffic data such as vehicle counts and system status are transmitted to the cloud platform for remote monitoring and analysis.

5. Wireless Communication (Wi-Fi)

Wi-Fi communication is enabled through the ESP32 module to transmit traffic statistics and system data to the cloud platform. This ensures seamless and real-time connectivity.

6. Email Automation (SMTP Protocol)

The system uses SMTP protocol in Python to automatically send violation evidence images with timestamps to a predefined email account. This reduces manual intervention and ensures proper documentation of traffic violation.

5.4 Coding

Aurdino Code:

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);
int red[4] = { 24, 27, 30, 33 }; int
yellow[4] = { 25, 28, 31, 34 }; int
green[4] = { 26, 29, 32, 35 };

int emergencyBtn =
22; int violationBtn =
23; int buzzer = 13;

String currentMode = "NORMAL"; int
emergencyLane = -1;

unsigned long previousMillis = 0;
int normalState = 0; int
currentLane = 0;

bool lastEmergencyState =
  HIGH; bool lastViolationState =
HIGH; String serialBuffer = "";

int emergencyCount = 0;
void setup() {

 lcd.init(); lcd.backlight();
 for (int i = 0; i < 4; i++) {
```

```
    pinMode(red[i],         OUTPUT);
    pinMode(yellow[i], OUTPUT);
    pinMode(green[i], OUTPUT);
   }

   pinMode(emergencyBtn, INPUT_PULLUP);
   pinMode(violationBtn,           INPUT_PULLUP);
   pinMode(buzzer,         OUTPUT);
   digitalWrite(buzzer, LOW);

   Serial.begin(9600);
   Serial2.begin(9600);

  lcd.setCursor(0,             0);
   lcd.print("Traffic                  System");
   delay(2000); lcd.clear();
  }

  void loop() {

   readButtons();
   readSerialNonBlocking();

   if (currentMode == "NORMAL") { normalModeNonBlocking();
   }
  } void
readButtons() {
bool
      emergencySt
ate   =
```

```
        digitalRead(e
mergencyBtn);
                bool violationState = digitalRead(violationBtn);
if (emergencyState != lastEmergencyState || violationState != lastViolationState) {


                  Serial.print("EMERGENCY:");
                  Serial.print(emergencyState == LOW ? 1 : 0); Serial.print(",VIOLATION:");
                  Serial.println(violationState == LOW ? 1 : 0);


                  lastEmergencyState = emergencyState;
                 lastViolationState = violationState; }
               }
void readSerialNonBlocking()
{ while (Serial.available()) {
char c = Serial.read(); if (c ==
'\n') {
                    serialBuffer.trim();
                    if (serialBuffer == "NORMAL") {
                      currentMode = "NORMAL";
                      emergencyLane = -1;
                      digitalWrite(buzzer, LOW); lcd.clear();
                    }


                    else if (serialBuffer.startsWith("CLEAR:")) {
                      emergencyCount++; currentMode = "EMERGENCY"; emergencyLane =
                            serialBuffer.substring(6).toInt() - 1;
                      activateEmergency(emergencyLane);
```

```
                String UNO = "$" + String(emergencyCount) + "b";
                 Serial2.println(UNO);
                }

                 serialBuffer = "";
               } else { serialBuffer
                += c;
               }
              }}

          void normalModeNonBlocking() { unsigned long
currentMillis = millis(); if (normalState == 0 && currentMillis -
previousMillis >= 1000) {

               setGreen(currentLane);



lcd.clear(); lcd.setCursor(0,        0);
               lcd.print("Green: Side ");
               lcd.print(currentLane + 1);

               previousMillis=        currentMillis; normalState = 1;
              } else if (normalState == 1 && currentMillis -
previousMillis >= 500) { setYellow(currentLane);
lcd.clear(); lcd.setCursor(0,        0);
               lcd.print("Yellow: Side ");
               lcd.print(currentLane + 1);

               previousMillis=        currentMillis; normalState = 0;
```

```
  currentLane++;      if
   (currentLane      >      3)      {
    currentLane = 0;
   }
 }}

void activateEmergency(int lane) {
 digitalWrite(buzzer, HIGH);

 for (int i = 0; i < 4; i++) {
  digitalWrite(red[i],    HIGH);
  digitalWrite(yellow[i], LOW);
  digitalWrite(green[i], LOW);
 }

 if (lane >= 0 && lane < 4) {

  digitalWrite(red[lane],     LOW);     digitalWrite(green[lane],
  HIGH);

  lcd.clear(); lcd.setCursor(0,
  0);
  lcd.print("EMERGENCY");
  lcd.setCursor(0,              1);
  lcd.print("Lane               ");
  lcd.print(lane +          1);
  lcd.print(" Clear");
 }
}
```

```
void setGreen(int lane) {

  for (int i = 0; i < 4; i++) {
    digitalWrite(red[i],    HIGH);
    digitalWrite(yellow[i], LOW);
    digitalWrite(green[i], LOW);
  }

  digitalWrite(red[lane], LOW);
digitalWrite(green[lane], HIGH); }

void setYellow(int lane) {
  digitalWrite(green[lane],     LOW);     digitalWrite(yellow[lane],
  HIGH);
}
```

PYTHON CODE for Violations & Detection:

```
from ultralytics import YOLO import cv2
import serial import time import smtplib
from        email.message          import
EmailMessage
arduino = serial.Serial('COM3', 9600, timeout=1) time.sleep(2)

ambulance_model          =          YOLO(r"c:\Users\mahen\Downloads\Traffic
Signals\violation\ambulance.v1i.yolov8\runs\detect\train\weights\best.pt")
violation_model          =          YOLO(r"c:\Users\mahen\Downloads\Traffic
Signals\violation\violation.v1i.yolov8\runs\detect\train\weights\best.pt")
```

```
MODE = None
last_sent =         ""
email_sent          =
    False lane_selected
    =
False     print("Waiting
for Arduino mode
command...")


EMAIL_ADDRESS = "embedded44@gmail.com"
EMAIL_PASSWORD = "alqplnjlmgbyoxst" TO_EMAIL =
"mahendrachintharaboina@gmail.com"

def send_violation_email(image_path):
    print("Email thread started") try:
        msg = EmailMessage() msg["Subject"] = "Camera
        Capture Alert" msg["From"] = EMAIL_ADDRESS
        msg["'To"] = TO_EMAIL msg.set_content("Image
        captured when IR detected")


        with open(image_path, "rb") as f:
            msg.add_attachment(
                f.read(),
                maintype="image",
                subtype="jpeg",
                filename=image_path
            )


        with smtplib.SMTP("smtp.gmail.com", 587, timeout=30) as smtp:
```

```
            smtp.ehlo() smtp.starttls()
            smtp.ehlo()
            smtp.login(EMAIL_ADDRESS,  EMAIL_PASSWORD)
            smtp.send_message(msg)

        print("    EMAIL SENT SUCCESSFULLY")

    except Exception as e:
        print("   EMAIL FAILED:", e)

cap = cv2.VideoCapture(1)

if          not          cap.isOpened():
    print("Camera   not   detected")
    exit()

while True:

    if   arduino.in_waiting   >   0:   cmd   =
        arduino.readline().decode().strip()

        if     "EMERGENCY:1"     in     cmd:
            MODE = "emergency"
            print("Emergency Mode Activated")

        elif "VIOLATION:1" in cmd: MODE
            = "violation" print("Violation
            Mode Activated")
```

```
            if MODE is None:
                continue

            ret, frame = cap.read() if
            not ret: break if MODE ==
            "emergency":

                results = ambulance_model(frame) ambulance_detected =
                False
for result in results: for box in result.boxes: class_id =
                    int(box.cls[0]) confidence = float(box.conf[0])
                    class_name = ambulance_model.names[class_id]

                     # Only detect if confidence > 90% if class_name.lower() ==
                       "ambulance" and confidence > 0.90: ambulance_detected =
                       True print(f"Ambulance Detected | Confidence:
                       {confidence:.2f}")

if ambulance_detected:

                    if not lane_selected: lane = input("Ambulance detected! Enter
                       lane to clear (1-4): ")

                       if lane in ["1", "2", "3", "4"]:
                         arduino.write(f"CLEAR:{lane}\n".encode())
                          print(f"Sent CLEAR:{lane} to Arduino") lane_selected
                          = True last_sent =
                          "Emergency"

                else:
```

```
            if last_sent != "Normal":
                arduino.write(b"NORMAL\n")

                print("Sent NORMAL to Arduino")
                lane_selected = False
                last_sent = "Normal"

            annotated = results[0].plot() cv2.imshow("Emergency Monitoring", annotated)

        # ====================================================

        # ⚠ VIOLATION MODE (EMAIL ONLY)
        # ====================================================
        elif MODE == "violation":

            results = violation_model(frame) violation_detected = False
for result in results: for box in result.boxes: class_id = int(box.cls[0]) confidence = float(box.conf[0])
                class_name = violation_model.names[class_id]
                if class_name.lower() == "violation" and confidence > 0.85: violation_detected = True print(f"Violation Detected | Confidence: {confidence:.2f}")

            if violation_detected:
                if not email_sent:
                print("Violation Detected!")

                image_path = "violation_capture.jpg" cv2.imwrite(image_path, frame) send_violation_email(image_path)
```

```
email_sent = True

            else:

                email_sent = False

            annotated = results[0].plot() cv2.imshow("Violation

            Monitoring", annotated)


        if  cv2.waitKey(1)      &       0xFF    ==
                ord('q'): break

    cap.release()                   cv2.destroyAllWindows()
    arduino.close()
```

CHAPTER-6

RESULTS

6.1 Results

The Traffic Violation Detection and Emergency Priority System was successfully implemented and tested under simulated traffic conditions. The system effectively detected traffic violations such as riding without a helmet and triple riding using the YOLOv8-based deep learning model integrated with a USB web camera. Upon detection of a violation, the system generated immediate visual alerts through LEDs, activated a buzzer for audible notification, and displayed violation details on the I2C LCD screen. Simultaneously, the captured evidence image with timestamp was automatically sent to a predefined email account, ensuring proper documentation and monitoring.

The Arduino Mega 2560 successfully controlled multi-lane traffic signals with accurate timing sequences, while the ESP32 microcontroller transmitted real-time vehicle count data to the ThingSpeak cloud platform. The cloud dashboard displayed traffic density information effectively, enabling remote monitoring and analysis. The emergency priority mechanism functioned as intended, providing uninterrupted green signals when the emergency push button was activated, thereby demonstrating improved response capability.

Overall, the system operated reliably, integrating computer vision, embedded control, IoT communication, and cloud monitoring into a unified intelligent traffic management framework. The experimental results indicate that the proposed system is scalable, cost-effective, and suitable for smart city traffic applications.

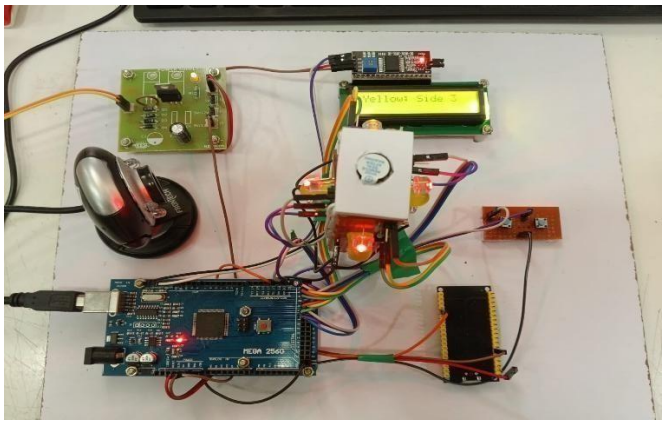

Fig 6.1.1: project set up

In **Fig.6.1.1** successfully detects traffic violations and gives priority to emergency vehicles. When an emergency vehicle is detected, the controller automatically changes the traffic signal and displays the priority message on the LCD ("Yellow Side"). The microcontroller processes inputs from sensors and controls the signals accordingly. This demonstrates effective automation for reducing delays and managing traffic violations.

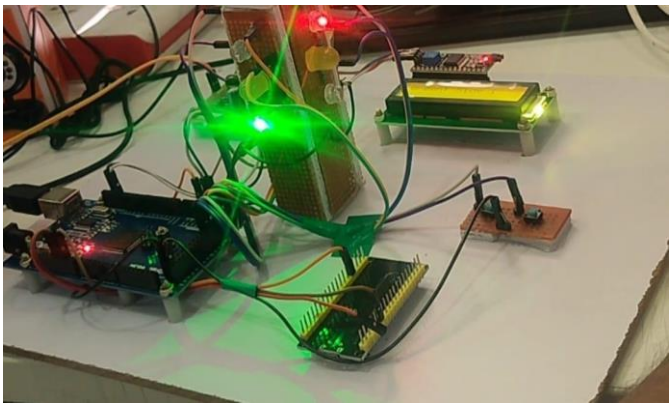

Fig 6.1.2: simulation of traffic lights

Fig.6.1.2 shows the Arduino Mega controls the connected modules and traffic signal LEDs. In the figure, the green LED is glowing, indicating that the traffic signal is allowing vehicles to pass in that direction. The LCD display shows the system status, while other modules handle communication and detection signals. This output demonstrates that the system successfully processes inputs and controls the traffic lights automatically.

Fig 6.1.3: Ambulance detection in different ways

Fig.6.1.3 show the detection of emergency vehicles by the proposed system. The model successfully identifies the ambulance in the input images and highlights it with a bounding box. The label “ambulance” along with the confidence score (such as 0.95) indicates that the system has recognized the vehicle as an emergency vehicle with high accuracy. This detection is performed using the trained image recognition model in the program. When an emergency vehicle is detected, the system sends a signal to the controller to give traffic priority by switching the signal to green. This helps the ambulance pass quickly through the traffic intersection

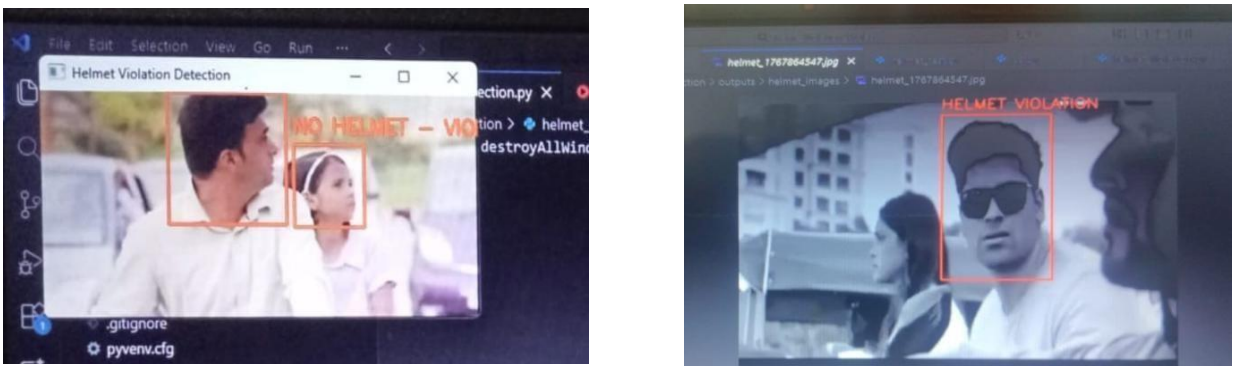

Fig 6.1.4: identify traffic violations

In **Fig 6.1.4** shows the triple riding detection result, where the system identifies more than two people on a motorcycle. The detection model places a bounding box around the riders and labels it as *triple riding*, indicating a traffic rule violation. shows the helmet detection result, where the system analyses the rider’s head and identifies whether a helmet is worn. When a

helmet is not detected, the system marks it as a helmet violation using a bounding box and label.

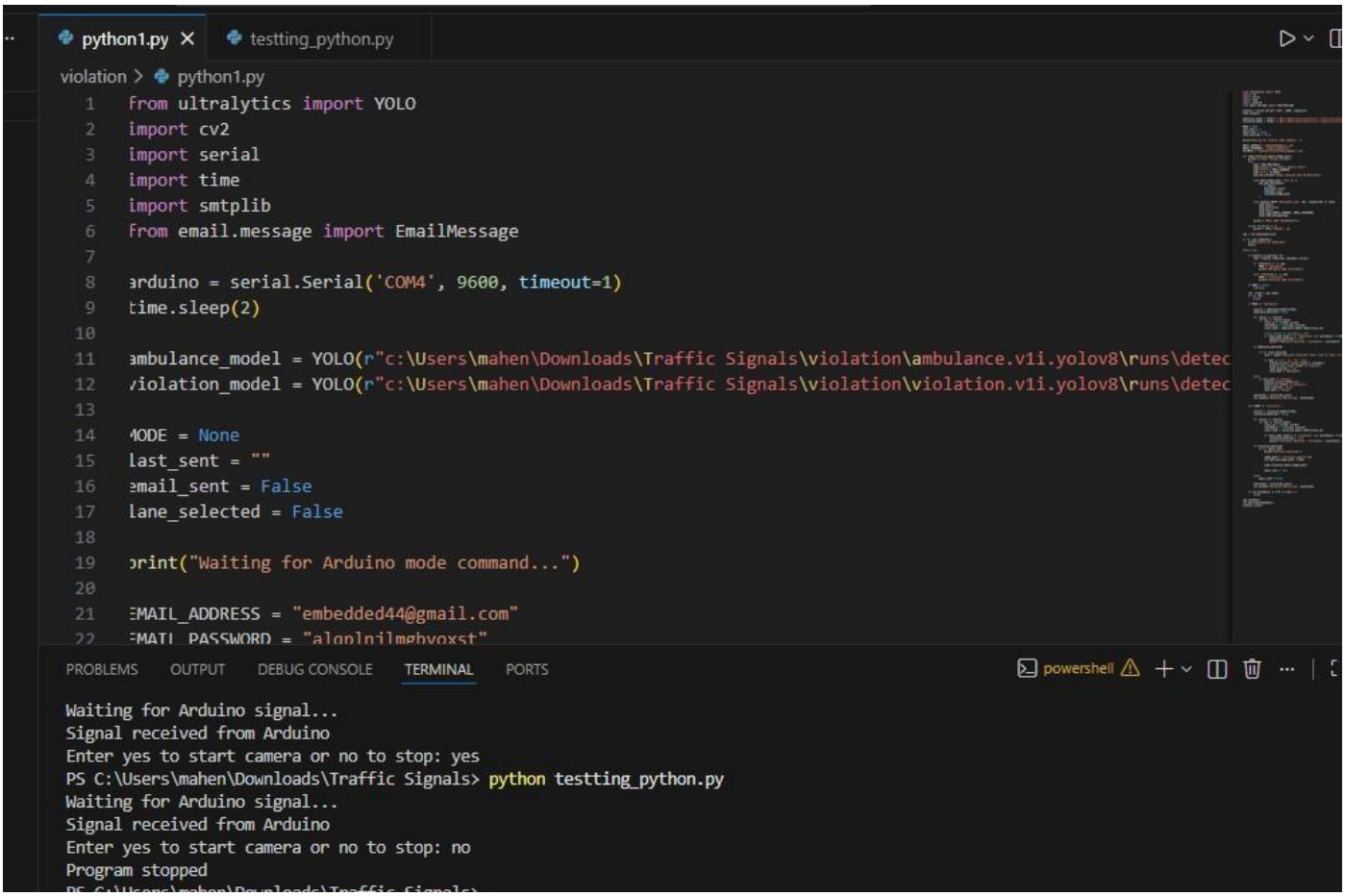

Fig 6.1.5: executed python code

In Fig 6.1.5 This code uses **YOLO (Ultralytics)** and **OpenCV** to detect traffic violations and emergency vehicles from video input. It connects to an **Arduino** through serial communication to receive signals for starting or stopping the system. The program loads trained models, processes video frames, and checks for violations. If needed, it can send alerts (like email) and control actions based on detection results.

6.2 Future Scope of Work

The Traffic Violation Detection and Emergency Priority System can be further enhanced and expanded to support advanced smart city applications. In the future, the system can be upgraded to detect additional traffic violations such as red-light jumping, over-speeding, lane violations, and illegal parking using advanced deep learning models. Integration with high-resolution IP cameras and edge AI devices can improve detection accuracy under diverse lighting and weather conditions.

The system can also be extended to implement automatic e-challan generation by integrating directly with government vehicle registration databases, enabling fully automated enforcement without manual intervention. Incorporating Automatic Number Plate Recognition (ANPR) with improved OCR accuracy can further strengthen vehicle identification reliability.

For better traffic optimization, adaptive signal control algorithms based on real-time traffic density and AI-based prediction models can be implemented. This would allow dynamic signal timing adjustments to reduce congestion and improve overall traffic flow efficiency.

The emergency priority mechanism can be enhanced using RFID tags, GPS tracking, or V2X (Vehicle-to-Everything) communication to automatically detect approaching emergency vehicles without requiring manual push-button activation.

Cloud integration can be expanded by incorporating advanced analytics dashboards, data visualization tools, and predictive traffic analysis using big data techniques. Mobile applications can also be developed for traffic authorities to monitor violations, receive alerts, and control signals remotely.

With further research and large-scale deployment, the proposed system has strong potential to evolve into a fully autonomous, AI-driven smart traffic management infrastructure suitable for modern urban environments.

CHAPTER-7

CONCLUSION

The Traffic Violation Detection and Emergency Priority System successfully demonstrate an intelligent and automated approach to modern traffic management by integrating computer vision, embedded systems, and IoT technologies. The system effectively detects traffic violations such as riding without a helmet and triple riding using a YOLOv8-based deep learning model, while simultaneously managing multilane traffic signals through an Arduino Mega. Real-time data communication via the ESP32 enables efficient cloud monitoring and documentation, including automated email alerts with evidence images and traffic flow analysis using ThingSpeak. Additionally, the emergency priority mechanism ensures uninterrupted green signals for emergency vehicles, improving response time and public safety. Overall, the proposed system offers a scalable, cost-effective, and reliable solution that reduces manual enforcement, enhances road safety, and supports the development of smart and responsive urban traffic infrastructures.

he Traffic Violation Detection and Emergency Priority System successfully demonstrates an intelligent and automated solution for modern traffic management by integrating computer vision, deep learning, embedded systems, and IoT technologies. The system effectively detects traffic violations such as riding without a helmet and triple riding using the YOLOv8-based deep learning model, ensuring real-time monitoring and improved road safety enforcement. Automated alert generation, LCD display updates, buzzer notifications, and email transmission of violation evidence reduce the need for manual supervision and enhance documentation efficiency. The Arduino Mega 2560 provides reliable multi-lane traffic signal control, while the ESP32 enables seamless cloud connectivity for real-time traffic data monitoring using the ThingSpeak platform. Additionally, the emergency priority mechanism ensures uninterrupted green signals for emergency vehicles, contributing to faster response times and improved public safety.

Overall, the proposed system offers a scalable, cost-effective, and reliable framework for smart urban traffic management. By combining automation, real-time analytics.

CHAPTER-8

FUTURE ENHANCEMENTS

1.Detection of Additional Traffic Violations

The system can be enhanced to detect more traffic violations such as red-light jumping, over-speeding, lane violations, illegal parking, and wrong-side driving. Advanced deep learning models can be trained to recognize multiple types of violations simultaneously, improving enforcement efficiency.

2.Automatic E-Challan Integration

Future versions of the system can be integrated directly with government vehicle registration databases to automatically generate and issue e-challans. This would eliminate manual verification and create a fully automated enforcement system.

3.Advanced ANPR (Automatic Number Plate Recognition)

Improving OCR accuracy and integrating a robust ANPR system will enhance vehicle identification under different lighting, weather, and camera angle conditions. Highresolution cameras can also be used to improve number plate detection reliability.

4.AI-Based Adaptive Traffic Signal Control

The traffic signal system can be upgraded with AI-based traffic prediction algorithms that dynamically adjust signal timing based on real-time vehicle density and historical traffic patterns. This will help reduce congestion and improve traffic flow efficiency.

5.Automatic Emergency Vehicle Detection

Instead of using a manual push button, the system can be enhanced with RFID, GPS tracking, or V2X (Vehicle-to-Everything) communication to automatically detect approaching emergency vehicles and provide signal priority.

6.Mobile Application for Monitoring

A mobile or web-based application can be developed for traffic authorities to monitor violations, view live traffic statistics, receive alerts, and control signals remotely in real time.

7.Edge AI Deployment

The deep learning model can be deployed on edge computing devices such as NVIDIA Jetson or Raspberry Pi with AI accelerators. This would reduce dependency on high-end computers and improve scalability.

8.Big Data Analytics & Predictive Modeling

Cloud data collected over time can be analyzed using big data analytics and machine learning techniques to predict traffic congestion patterns and optimize city-wide traffic management strategies.

9.Integration with Smart City Infrastructure

The system can be integrated with other smart city components such as smart parking systems, public transport monitoring, and surveillance networks to create a unified intelligent transportation ecosystem.

CHAPTER-9

APPENDIX / APPENDICES

Appendix A: System Architecture Details

This appendix defines the architectural framework supporting the **AI-Powered Traffic**

Violation Detection and Emergency Priority Vehicle System

The system integrates deep learning-based computer vision, embedded control systems, and IoT cloud communication with a network of traffic signal devices and monitoring components. The architecture utilizes a USB camera for live traffic monitoring, a YOLOv8-based detection model for violation identification, an Arduino Mega 2560 for traffic signal control, and an ESP32 module for cloud communication.

The system enables automated detection of traffic violations such as riding without a helmet and triple riding using real-time video processing. Upon detecting a violation, commands are processed through the central processing unit (Python-based AI module) and transmitted to the microcontroller, which executes appropriate control actions such as activating alerts, updating the display, and managing traffic signals. Simultaneously, traffic data is transmitted to the cloud platform for remote monitoring and analytics. The emergency priority mechanism allows manual override of traffic signals to provide uninterrupted green signals for emergency vehicles.

Component Description

1. Traffic Monitoring Device (Camera Module):

Represents the live video input system. It captures real-time traffic footage and sends video frames to the AI processing module for analysis.

2. AI Processing Module (YOLOv8 + Python):

Processes video frames using deep learning algorithms to detect traffic violations such as helmet non-compliance and triple riding. Generates detection results and triggers further actions.

3. Violation Management Module:

Captures evidence images, generates timestamps, and initiates automated email notifications to authorities for documentation.

4. Traffic Control Unit (Arduino Mega 2560):

Executes control commands received from the AI module. It manages traffic signal sequencing, LED indicators, LCD display updates, buzzer alerts, and emergency override logic.

5. IoT Communication Module (ESP32):

Transmits traffic data such as vehicle count and system status to the ThingSpeak cloud platform using Wi-Fi connectivity for real-time monitoring.

6. Power Supply Unit:

Manages and regulates electrical power distribution to the microcontroller, sensors, LEDs, and other hardware components to ensure stable and safe operation.

7. Emergency Priority Module (Push Button Interface):

Allows activation of emergency vehicle priority mode, temporarily overriding normal signal timing to provide uninterrupted green signals.

Appendix B: Technical Specifications

This section defines the technical configuration and operational parameters of the **AIPowered Traffic Violation Detection and Emergency Priority Vehicle System** . The system integrates embedded controllers, deep learning-based computer vision, IoT communication modules, and traffic control hardware to provide automated violation detection and emergency signal management. The architecture consists of a vision processing unit, control unit, communication unit, display and alert modules, and regulated power supply components.

The system operates using a USB web camera for live video acquisition, a Python-based YOLOv8 model for violation detection, an Arduino Mega 2560 for signal control

logic, and an ESP32 module for real-time cloud communication. The system is designed for real- time processing, scalable deployment, and cost-effective implementation in urban traffic environments.

Component Specification

• **Vision Processing Unit:**

Utilizes Python, OpenCV, and YOLOv8 deep learning model for real-time detection of helmet violations and triple riding. Supports automatic image capture and OCR-based number plate recognition.

• **Microcontroller Unit (Arduino Mega 2560):**

Operating Voltage: 5V

Input Voltage Range: 7–12V

Digital I/O Pins: 54

Analog Inputs: 16

Clock Speed: 16 MHz

Flash Memory: 256 KB

Controls traffic signals, buzzer alerts, LCD display, and emergency override logic.

• **IoT Communication Module (ESP32):**

Dual-core processor (160–240 MHz)

Integrated Wi-Fi (802.11 b/g/n)

Bluetooth support

External Flash: 4MB

Transmits traffic data to Thing Speak cloud platform for monitoring and analytics.

• **Display Module (16×2 I2C LCD):**

Operating Voltage: 5V

Interface: I2C

Displays violation messages and system status.

• **Alert Module (Buzzer & LEDs):**

Buzzer Operating Voltage: 5V DC

LEDs: Red, Yellow,Green (multi-lane signal representation)
Provides visual and audible violation indication.

• **Emergency Priority Module:**

Push-button activated override mechanism enabling uninterrupted green signal for emergency vehicles.

• **Power Supply Unit:**

Transformer-based AC to DC conversion

Rectifier for AC to DC conversion

Capacitors for filtering

Voltage Regulators (7805 / 7812) for stable 5V and 12V output

Operating Requirements

• Power Supply: 7–12V DC

• Internet Connectivity required for cloud and email services

• Operating Temperature: 0°C to 50°C

• Compatible with Windows-based system for Python execution

Appendix C: User Manual Excerpt

• Internet Connectivity required for cloud and email services

• Operating Temperature: 0°C to 50°C

• Compatible with Windows-based system for Python execution

User Manual Excerpt

This section provides operational guidance for users and traffic authorities handling the AI-Powered Traffic Violation Detection and Emergency Priority Vehicle System

System Initialization

• Connect power supply to Arduino and ESP32 modules.

• Connect USB web camera to processing computer.

• Ensure Wi-Fi connectivity for cloud services.

• Launch the Python-based detection application.

• LCD displays “System Initialized”.

• Traffic signal operation begins automatically.

Violation Detection Operation

- The camera continuously captures traffic footage.
- YOLOv8 model processes frames in real time.
- On detecting helmet violation or triple riding:
- LCD displays violation message.
- Buzzer activates.
- Evidence image captured with timestamp.
- Email notification sent automatically.

Emergency Priority Operation

• Press the emergency push button.

• The selected lane receives uninterrupted green signal.

• After release, system resumes normal signal cycle.

Cloud Monitoring

• Traffic data is transmitted to Thing Speak platform.

• Authorities can access vehicle count statistics remotely.

• Real-time monitoring dashboard displays traffic flow information.

Safety & Maintenance.

• Ensure regulated voltage supply (7–12V).

• Avoid moisture exposure.

• Maintain stable internet connection.

• Periodically check camera alignment and system wiring.

Appendix D: Sample Operational Log

This appendix documents a sample operational log demonstrating system performance under normal operating conditions.

Introduction

The log records system commands and responses, providing insight into usage patterns and overall performance.

Performance Notes

- All commands executed within minimal response time.
- No system errors observed.
- Network connectivity remained stable throughout operation.

Planned Actions

- Schedule appliance automation based on user preferences.
- Review daily energy consumption through the system dashboard.

Appendix E: Integration Guidelines

Device Compatibility Check

Before integrating any new hardware module or IoT device into the system, compatibility verification must be performed to ensure smooth operation with the existing architecture (Arduino Mega 2560, ESP32, Camera, Cloud Platform).

Electrical Compatibility

- Device operating voltage must match system levels:
    - **5V** (Arduino Mega peripherals)
    - **3.3V** (ESP32 logic level)
- Current consumption must not exceed controller limits.
- Use level shifters if interfacing 5V and 3.3V devices.
- Ensure proper grounding to avoid signal instability.

Software Compatibility

- Library support in Arduino IDE (Embedded C).
- Python driver/library support (if used with camera/AI module).
- Firmware should be updated to latest stable version.
- API documentation must be available. **Software Compatibility**
- Library support in Arduino IDE (Embedded C).
- Python driver/library support (if used with camera/AI module).
- Firmware should be updated to latest stable version.
- API documentation must be available.

Functional Compatibility

The device must align with system objectives:

- Traffic monitoring
- Violation detection

- Emergency vehicle prioritization
- Data logging

  The device should not introduce processing delay in real-time operations.

Network Configuration

Network configuration ensures proper communication between ESP32, cloud server, and monitoring interface. Wi-Fi Setup • Configure Wi-Fi credentials in ESP32 firmware.

- Use secure WPA2 network.
- Ensure 2.4 GHz network compatibility (ESP32 standard).
- Assign Static IP (recommended for stability).

CHAPTER-10

References / Bibliography

[1] A. P. S. Agre, S. S. Deokar, G. B. Kunjir, and T. Nalawade,

"AI-Powered Helmet and Vehicle Number Plate Recognition System with Automatic Traffic Violation Detection and E-Challan Generation," *International Journal of Engineering Research & Technology (IJERT)*, 2023.

[2] B. M. Bhavana, S. Sathyanarayana, S. M. K. Bhavana, G. N. Nisarga, and S. S. Harshitha,

"Traffic Eye: A Literature Survey on AI-Powered Traffic Rules Violation Detection System," *International Journal of Scientific Research in Computer Science*, 2022.

[3] A. Hajare, L. Sonawane, U. Patel, and L. Sonawane,

"AI and IoT Based Traffic Signal and Rule Enforcement System," *Proceedings of International Conference on Smart Computing and Communication*, 2023.

[4] Conference Authors,

"AI and IoT Based Traffic Signal and Rule Enforcement System," *IEEE International Conference on Intelligent Transportation Systems*, 2022.

[5] Conference Authors,

"Real-Time Multi-class Helmet Violation Detection Using YOLOv8 with License Plate Recognition," *International Conference on Artificial Intelligence and Data Science*, 2023.

[6] M. N. Chan and T. Thuzar,

"Vehicle Detection and Traffic Violation Parking Management System," *International Journal of Computer Applications*, vol. 185, no. 24, 2024.

[7] P. Siva, G. B. Pujitha, and G. S. Krishna,

"Dynamic Context Capture Using Multimodal Vision Systems," *IEEE Access*, 2020.

[8] (For YOLOv8 Reference)

G. Jocher et al.,

"YOLOv8: Ultralytics Real-Time Object Detection," Ultralytics, 2023. [Online].

Available: https://docs.ultralytics.com

[9] R. G. Putra, W. R. Pribadi, D. J. Irawono, and E. J. Sudarman,

"Adaptive Traffic Light Control Using Computer Vision and Vehicle Density Algorithm," *International Journal of Advanced Computer Science and Applications*, 2021.

[10] X. Shen, J. Li, Y. Fan, and P. Guo,

"Traffic Violation Alert Generation System Using CNN-Based Feature Extraction and Temporal Video Analysis," *IEEE Transactions on Intelligent Transportation Systems*, 2023.